

ΑΡΙΣΤΟΤΕΛΙΟ ΠΑΝΕΠΗΣΤΙΜΙΟ
Τμήμα Πληροφορικής

Mini Project: RML Mapping + RDF4J + GraphDB

Αναφορά Υλοποίησης Java Pipeline

Dataset: Spotify & YouTube
Repository GraphDB: MiniProject

Ακαδημαϊκό έτος 2025–2026

Περιεχόμενα

1. Εισαγωγή.....	1
2. Επισκόπηση της αρχιτεκτονικής.....	1
3. Η κλάση Main - Συντονισμός του pipeline.....	1
3.1 Βασικές σταθερές.....	1
3.2 Ροή εκτέλεσης.....	1
4. Στάδιο 1 - CsvPreprocessor (Προγραμματιστική προεπεξεργασία).....	1
4.1 Κατηγορίες προβλημάτων που διορθώνονται.....	1
4.2 Δομή και λειτουργία.....	1
5. Στάδιο 2 - RmlRunner (Μετασχηματισμός σε RDF).....	1
5.1 Δημιουργία της εξωτερικής διεργασίας.....	1
5.2 Παρακολούθηση και έλεγχος εκτέλεσης.....	1
6. Στάδιο 3 - GraphDbLoader (Φόρτωση & Reasoning).....	1
6.1 Σύνδεση και καθαρισμός.....	1
6.2 Μετατροπή της οντολογίας OWL/XML → Turtle.....	1
6.3 Φόρτωση δεδομένων και reasoning.....	1
7. Στάδιο 4 - SparqlRunner (SPARQL ερωτήματα).....	1
7.1 Δομή της κλάσης.....	1
7.2 Τα έξι ερωτήματα.....	1
7.3 Υποστήριξη reasoning.....	1
8. Δομή του project και ρυθμίσεις (pom.xml).....	1
8.1 Εξαρτήσεις.....	1
9. Οδηγίες εγκατάστασης και εκτέλεσης.....	1
10. Συμπεράσματα.....	1

1. Εισαγωγή

Η παρούσα αναφορά περιγράφει το τμήμα της εργασίας που αφορά την υλοποίηση σε Java του ολοκληρωμένου pipeline μετασχηματισμού και αποθήκευσης δεδομένων σε RDF γράφο. Στόχος του pipeline είναι να πάρει ένα tabular dataset (σε μορφή CSV), να το καθαρίσει προγραμματιστικά, να το μετατρέψει σε RDF/Turtle με χρήση RML κανόνων, να φορτώσει τα αποτελέσματα μαζί με την οντολογία σε ένα repository της GraphDB όπου ενεργοποιείται και ο reasoner, και τελικά να εκτελέσει μια σειρά από SPARQL ερωτήματα που αναδεικνύουν την προστιθέμενη αξία της αναπαράστασης σε RDF.

Το dataset που χρησιμοποιήθηκε είναι το «Spotify and YouTube», το οποίο συσχετίζει μουσικά κομμάτια (tracks) με καλλιτέχνες, άλμπουμ, χαρακτηριστικά ήχου (όπως danceability και energy) και μετρικές απήχησης από το YouTube (views, likes, comments). Η ύπαρξη αυτών των σχέσεων μεταξύ οντοτήτων είναι που δίνει νόημα στον μετασχηματισμό σε RDF και επιτρέπει την εξαγωγή νέων συμπερασμάτων μέσω της οντολογίας.

Η υλοποίηση οργανώνεται σε πέντε κλάσεις Java που εκτελούν τα διαδοχικά στάδια του pipeline, καθώς και στο αρχείο ρυθμίσεων pom.xml του Maven. Στις ενότητες που ακολουθούν περιγράφεται αναλυτικά ο ρόλος κάθε στοιχείου.

2. Επισκόπηση της αρχιτεκτονικής

Το pipeline ακολουθεί μια γραμμική, σαφώς διαχωρισμένη ροή τεσσάρων σταδίων. Κάθε στάδιο παράγει ένα αρχείο ή κατάσταση που τροφοδοτεί το επόμενο, ώστε τα στάδια να είναι αποσυνδεδεμένα μεταξύ τους και εύκολα ελέγξιμα:

Στάδιο	Κλάση	Είσοδος	Έξοδος
1. Προεπεξεργασία	CsvPreprocessor	Spotify_Youtube.csv	Spotify_Youtube_clean.csv
2. Μετασχηματισμός RML	RmlRunner	mapping.ttl + clean CSV	output.ttl
3. Φόρτωση & Reasoning	GraphDbLoader	ontology.owlx + output.ttl	GraphDB repository
4. Ερωτήματα	SparqlRunner	GraphDB endpoint	Αποτελέσματα στην κονσόλα

Συντονιστής όλων των σταδίων είναι η κλάση Main, η οποία λειτουργεί ως orchestrator: καλεί διαδοχικά τις υπόλοιπες κλάσεις, περνώντας τους τα σωστά paths και ρυθμίσεις. Σημειώνεται ότι δύο από τα στάδια αποτελούν προγραμματιστικούς μετασχηματισμούς που λειτουργούν συμπληρωματικά με τους RML κανόνες: ο καθαρισμός του CSV (CsvPreprocessor) και η μετατροπή της οντολογίας από OWL/XML σε Turtle (μέσα στον GraphDbLoader).

3. Η κλάση Main - Συντονισμός του pipeline

Η κλάση Main αποτελεί το σημείο εκκίνησης (entry point) της εφαρμογής. Ο ρόλος της είναι να εκτελέσει με τη σωστή σειρά όλα τα βήματα της διαδικασίας και να παραδώσει σε κάθε κλάση τα κατάλληλα ορίσματα.

3.1 Βασικές σταθερές

```
static final Path DATA_DIR = Paths.get("").toAbsolutePath().getParent();
static final String GRAPHDB_URL = "http://localhost:7200";
static final String REPO_NAME = "MiniProject";
```

Η σταθερά DATA_DIR δείχνει στον γονικό φάκελο του pipeline, δηλαδή στον βασικό φάκελο του project όπου βρίσκονται τα αρχεία δεδομένων. Το GRAPHDB_URL ορίζει τη διεύθυνση του τοπικού GraphDB server, ενώ το REPO_NAME καθορίζει το repository με όνομα MiniProject, όπως απαιτεί η εκφώνηση.

3.2 Ροή εκτέλεσης

Η μέθοδος main εκτελεί τα τέσσερα στάδια διαδοχικά:

1. Καλεί τον CsvPreprocessor για να καθαρίσει το αρχικό CSV.
2. Καλεί τον RmlRunner για να παραχθεί το RDF αρχείο output.ttl.
3. Καλεί τον GraphDbLoader για να φορτωθούν οντολογία και RDF δεδομένα στην GraphDB.
4. Καλεί τον SparqlRunner για να εκτελεστούν τα SPARQL ερωτήματα.

Σε κάθε βήμα εμφανίζονται ενημερωτικά μηνύματα στην κονσόλα και, με την επιτυχή ολοκλήρωση όλων των σταδίων, τυπώνεται το μήνυμα «=== Pipeline complete ===». Η εφαρμογή ξεκινά με την εντολή `mvn exec:java`.

4. Στάδιο 1 - CsvPreprocessor (Προγραμματιστική προεπεξεργασία)

Η κλάση CsvPreprocessor καθαρίζει το αρχικό αρχείο Spotify_Youtube.csv πριν αυτό περάσει στον RML Mapper. Πρόκειται για έναν προγραμματιστικό μετασχηματισμό που υλοποιήθηκε σκόπιμα εκτός των RML κανόνων, επειδή αφορά διορθώσεις τιμών που δεν μπορούν να εκφραστούν εύκολα σε επίπεδο mapping και που, αν έμεναν ανεπεξέργαστες, θα προκαλούσαν σφάλματα στους τύπους δεδομένων του RDF/XML Schema.

4.1 Κατηγορίες προβλημάτων που διορθώνονται

Boolean τιμές. Οι στήλες Licensed και official_video περιέχουν τιμές όπως True/False, οι οποίες μετατρέπονται σε πεζά (true/false) ώστε να είναι συμβατές με τον τύπο `xsd:boolean`.

Αριθμητικές τιμές τύπου long. Οι στήλες Stream, Likes, Comments και Views εμφανίζονται συχνά ως δεκαδικοί (π.χ. 693555221.0) ενώ πρόκειται για ακραίους. Μετατρέπονται σε ακέραιους (693555221) για να χρησιμοποιηθούν σωστά ως `xsd:long`.

Κενές ή μη έγκυρες τιμές. Κελιά που είναι null, κενά ή περιέχουν τη συμβολοσειρά «nan» αντικαθίστανται με κενό string, ώστε το mapping να τα αγνοεί αντί να παράγει σφάλμα.

4.2 Δομή και λειτουργία

Οι στήλες που χρειάζονται ειδική επεξεργασία ορίζονται ως σύνολα (sets):

```
private static final Set<String> BOOL_COLS = Set.of("Licensed", "official_video");
private static final Set<String> LONG_COLS = Set.of("Stream", "Likes", "Comments",
"Views");
```

Η μέθοδος `process(Path input, Path output)` ανοίγει το αρχικό CSV με `Reader` και δημιουργεί το καθαρισμένο CSV με `Writer`, μέσα σε δομή `try-with-resources` ώστε τα αρχεία να κλείνουν αυτόματα. Με χρήση της βιβλιοθήκης `Apache Commons CSV` διαβάζονται τα `headers`, και η επεξεργασία γίνεται με βάση το όνομα κάθε στήλης αντί της θέσης της. Για κάθε γραμμή και κάθε στήλη εφαρμόζεται ένας από τους τέσσερις κανόνες (κενές τιμές, `boolean`, `long`, ή διατήρηση ως έχει), και το αποτέλεσμα γράφεται με τον `CSVPrinter`.

Σημαντική παρατήρηση: εντοπίστηκε ότι ορισμένες τιμές περιγραφών δήλωναν εσφαλμένα `literal datatype` στην παραγόμενη `RDF/Turtle`, οδηγώντας σε σφάλματα τύπου «*invalid typed literals*». Η αφαίρεσή τους στο στάδιο της προεπεξεργασίας ήταν απαραίτητη για την ομαλή λειτουργία του `mapping`.

Συνολικά, ο `CsvPreprocessor` είναι ένα αμιγώς προπαρασκευαστικό στάδιο: δεν εκτελεί `mapping`, δεν συνδέεται με την `GraphDB` και δεν παράγει `RDF`. Ο ρόλος του είναι να εγγυηθεί ότι τα δεδομένα που φτάνουν στον `RML Mapper` είναι έγκυρα και ασφαλή.

5. Στάδιο 2 - `RmlRunner` (Μετασχηματισμός σε `RDF`)

Η κλάση `RmlRunner` είναι υπεύθυνη για την εκτέλεση του `RML Mapper`. Δεν υλοποιεί η ίδια τη λογική του `mapping`: αντ' αυτού λειτουργεί ως ενδιάμεσος που καλεί το έτοιμο `rmlmapper.jar` ως εξωτερική διεργασία, εφαρμόζοντας τους `RML` κανόνες του αρχείου `mapping.ttl` πάνω στο καθαρισμένο CSV και παράγοντας το αρχείο `output.ttl`.

5.1 Δημιουργία της εξωτερικής διεργασίας

```
ProcessBuilder pb = new ProcessBuilder(
    "java", "-jar", jar.toAbsolutePath().toString(),
    "-m", mapping.toAbsolutePath().toString(),
    "-o", output.toAbsolutePath().toString(),
    "-s", "turtle"
);
pb.directory(workDir.toFile());
pb.redirectErrorStream(true);
```

Η κλάση κατασκευάζει και εκτελεί την εντολή `java -jar rmlmapper.jar -m mapping.ttl -o output.ttl -s turtle`. Τα ορίσματα ορίζουν το `mapping file` (`-m`), το αρχείο εξόδου (`-o`) και τη μορφή εξόδου `Turtle` (`-s turtle`).

Φάκελος εργασίας. Η κλήση `pb.directory(workDir.toFile())` ορίζει τον φάκελο εργασίας ώστε ο `mapper` να εντοπίζει σωστά τα αρχεία που αναφέρονται με σχετικά `paths` μέσα στο `mapping.ttl`, όπως το `Spotify_Youtube_clean.csv`, αποφεύγοντας έτσι σφάλματα διαδρομών.

5.2 Παρακολούθηση και έλεγχος εκτέλεσης

Με την εντολή `redirectErrorStream(true)` το `standard output` και το `standard error` ενοποιούνται σε ένα κοινό `stream`. Η κλάση διαβάζει γραμμή-γραμμή τα `logs` του `mapper` και τα τυπώνει με το πρόθεμα `[rmlmapper]`, ώστε να ξεχωρίζουν από τα υπόλοιπα μηνύματα του `pipeline`. Στη συνέχεια, με `waitFor()` αναμένει την ολοκλήρωση της διεργασίας και ελέγχει τον `exit code`: αν είναι διαφορετικός του μηδενός, εγείρεται `RuntimeException` και το `pipeline` σταματά αμέσως. Διαφορετικά, εμφανίζεται μήνυμα επιτυχίας.

6. Στάδιο 3 - GraphDbLoader (Φόρτωση & Reasoning)

Η κλάση GraphDbLoader αναλαμβάνει τη φόρτωση της οντολογίας και των RDF δεδομένων στην GraphDB. Πριν τη φόρτωση καθαρίζει το repository, ώστε κάθε εκτέλεση του pipeline να ξεκινά από καθαρή βάση και να μην κρατούνται δεδομένα από προηγούμενα runs.

6.1 Σύνδεση και καθαρισμός

Η σύνδεση πραγματοποιείται μέσω της κλάσης HTTPRepository του RDF4J, η οποία επιτρέπει απομακρυσμένη πρόσβαση στο repository μέσω HTTP. Το base URI που χρησιμοποιείται κατά τη φόρτωση είναι `http://purl.org/ontology/mo/`, δηλαδή το namespace του Music Ontology. Μέσω ενός ενεργού RepositoryConnection εκτελείται πρώτα η μέθοδος `conn.clear()` που διαγράφει τα υπάρχοντα δεδομένα.

6.2 Μετατροπή της οντολογίας OWL/XML → Turtle

Δεύτερος προγραμματιστικός μετασχηματισμός του pipeline. Η βοηθητική μέθοδος `owxToTurtle` χρησιμοποιεί το OWL API για να φορτώσει την οντολογία από το αρχείο `ontology.owx` και να την αποθηκεύσει σε μορφή Turtle, εξ ολοκλήρου στη μνήμη, χωρίς δημιουργία ενδιάμεσου αρχείου στον δίσκο.

```
OWLOntologyManager mgr = OWLManager.createOWLOntologyManager();
OWLOntologyLoaderConfiguration config = new OWLOntologyLoaderConfiguration()
    .setMissingImportHandlingStrategy(MissingImportHandlingStrategy.SILENT);
mgr.setOntologyLoaderConfiguration(config);
OWLOntology ontology = mgr.loadOntologyFromOntologyDocument(owxFile.toFile());
ByteArrayOutputStream baos = new ByteArrayOutputStream();
mgr.saveOntology(ontology, new TurtleDocumentFormat(), baos);
return baos.toString(StandardCharsets.UTF_8);
```

Η ρύθμιση `MissingImportHandlingStrategy.SILENT` εξασφαλίζει ότι, αν κάποια εξωτερική imported οντολογία δεν είναι προσβάσιμη, η φόρτωση δεν αποτυγχάνει αλλά συνεχίζεται κανονικά.

6.3 Φόρτωση δεδομένων και reasoning

Στη συνέχεια φορτώνονται διαδοχικά η οντολογία (ως string Turtle μέσω `StringReader`) και το αρχείο RDF δεδομένων `output.ttl` που παρήγαγε ο RML Mapper. Η ταυτόχρονη παρουσία οντολογίας και δεδομένων στο ίδιο repository, σε συνδυασμό με τον ενεργοποιημένο reasoner της GraphDB, είναι αυτό που επιτρέπει την εξαγωγή των inferred κλάσεων (π.χ. `DanceableTrack`, `LongTrack`, `LicencedTrack`). Τέλος, η κλάση τυπώνει το συνολικό πλήθος statements μέσω `conn.size()` ως γρήγορο έλεγχο ορθής φόρτωσης, και κλείνει σωστά το repository σε `finally` block.

7. Στάδιο 4 - SparqlRunner (SPARQL ερωτήματα)

Η κλάση SparqlRunner συνδέεται στο SPARQL endpoint της GraphDB και εκτελεί μια σειρά από έξι προκαθορισμένα SELECT ερωτήματα (Q1-Q6). Τα ερωτήματα δοκιμάστηκαν πρώτα μέσα στο περιβάλλον της GraphDB και, αφού επέστρεψαν σωστά αποτελέσματα, ενσωματώθηκαν στον κώδικα για την ολοκλήρωση του pipeline. Τα αποτελέσματα εμφανίζονται μορφοποιημένα στην κονσόλα.

7.1 Δομή της κλάσης

- ένα κοινό block από prefixes (PREFIXES) που χρησιμοποιείται σε όλα τα queries,
- έξι σταθερά queries (Q1–Q6),
- μια λίστα QUERIES που συνδυάζει περιγραφή και query,
- τη μέθοδο `run(String sparqlEndpoint)` που εκτελεί τα queries,
- και τη βοηθητική μέθοδο `formatRow(...)` για την ευανάγνωστη εκτύπωση των αποτελεσμάτων.

7.2 Τα έξι ερωτήματα

Query	Περιγραφή
Q1 - Top 20 DanceableTracks	Τα 20 tracks με τη μεγαλύτερη danceability που ανήκουν στην inferred κλάση <code>mo:DanceableTrack</code> .
Q2 - FamousDanceTracks	Tracks της inferred κλάσης <code>mo:FamousDanceTrack</code> , δηλαδή ταυτόχρονα danceable και popular.
Q3 - Artists with most LicencedTracks	Πλήθος licensed tracks ανά καλλιτέχνη, βάσει της inferred κλάσης <code>mo:LicencedTrack</code> .
Q4 - DanceableTrack & EnergizedTrack	Καλλιτέχνες με τουλάχιστον ένα track σε καθεμία από τις κλάσεις <code>mo:DanceableTrack</code> και <code>mo:EnergizedTrack</code> .
Q5 - Μέσοι όροι ανά album	Μέσος όρος danceability και energy, καθώς και πλήθος tracks, ανά album.
Q6 - LongTracks & YouTube engagement	Tracks μεγάλης διάρκειας (<code>mo:LongTrack</code>) με βάση likes και comments από το YouTube.

Τα ερωτήματα είναι σχεδιασμένα ώστε να μην είναι τετριμμένα: αξιοποιούν τη διασύνδεση δεδομένων (`tracks ↔ artists ↔ albums`), τη χρήση της domain οντολογίας και κυρίως τις inferred κλάσεις, αναδεικνύοντας έτσι τα πλεονεκτήματα της αναπαράστασης σε RDF.

7.3 Υποστήριξη reasoning

Ιδιαίτερα σημαντικό στοιχείο είναι η ενεργοποίηση των inferred αποτελεσμάτων με την εντολή `query.setIncludeInferred(true)`. Χάρη σε αυτή, στα αποτελέσματα περιλαμβάνονται και οι αυτόματες ταξινομήσεις που προκύπτουν από την οντολογία (π.χ. η ταξινόμηση ενός track ως `DanceableTrack`, `PopularTrack`, `LongTrack` ή `LicencedTrack`), και όχι μόνο τα ρητά δηλωμένα statements.

Τεχνικά, η μέθοδος `run` δημιουργεί ένα `HTTPRepository`, το αρχικοποιεί, αποκτά `RepositoryConnection` και για κάθε query κατασκευάζει ένα `TupleQuery`, ενεργοποιεί τα inferred αποτελέσματα, το εκτελεί, διαβάζει τις γραμμές και τις εμφανίζει σε μορφή πίνακα μέσω της `formatRow`, η οποία περιορίζει το μήκος μεγάλων strings και στοιχίζει τις στήλες για ευκρίνεια. Στο τέλος κλείνει το repository.

8. Δομή του project και ρυθμίσεις (pom.xml)

Το αρχείο `pom.xml` ορίζει την ταυτότητα του project, τις εξαρτήσεις, την έκδοση της Java και τον τρόπο μεταγλώττισης και εκτέλεσης. Το project ταυτοποιείται με `groupId gr.unipi`, `artifactId ontology-pipeline`, `version 1.0-SNAPSHOT` και `packaging jar`. Ο κώδικας μεταγλωττίζεται με Java 17 και encoding UTF-8.

8.1 Εξαρτήσεις

Βιβλιοθήκη	Ρόλος στο pipeline
RDF4J Client	Σύνδεση με το GraphDB/RDF4J repository, εκτέλεση SPARQL queries και χειρισμός RDF δεδομένων.
OWL API (owlapi-distribution)	Ανάγνωση της οντολογίας σε OWL/XML και μετατροπή της σε Turtle πριν τη φόρτωση.
Apache Commons CSV	Ανάγνωση και προεπεξεργασία/καθαρισμός του αρχικού CSV dataset.
SLF4J Simple	Βασικό logging κατά την εκτέλεση, για καθαρότερη εμφάνιση μηνυμάτων.

Οι εκδόσεις των βιβλιοθηκών RDF4J διαχειρίζονται κεντρικά μέσω του `rdf4j-bom` στο `dependencyManagement`, ώστε να αποφεύγονται ασυμβατότητες. Για το `build` χρησιμοποιούνται το `maven-compiler-plugin` (μεταγλώττιση με Java 17) και το `exec-maven-plugin`, που επιτρέπει την εκτέλεση της εφαρμογής με την εντολή `mvn exec:java`, με κύρια κλάση την `gr.unipi.pipeline.Main`.

9. Οδηγίες εγκατάστασης και εκτέλεσης

Για την εκτέλεση του pipeline ακολουθούνται τα παρακάτω βήματα:

1. Επιβεβαίωση ότι υπάρχει εγκατεστημένο JDK (έλεγχος με `java -version`).
2. Εγκατάσταση του Apache Maven (έκδοση 3.9.x).
3. Ορισμός της μεταβλητής περιβάλλοντος `MAVEN_HOME` ώστε να δείχνει στον φάκελο του Maven και προσθήκη του `%MAVEN_HOME%\bin` στο Path (τόσο στα user όσο και στα system variables).
4. Επανεκκίνηση του terminal και έλεγχος εγκατάστασης με `mvn -version`.
5. Εκκίνηση της GraphDB και δημιουργία repository με όνομα `MiniProject`.
6. Μετάβαση στον φάκελο του project και εκτέλεση των εντολών `mvn compile` και έπειτα `mvn exec:java`.

Με την ολοκλήρωση των παραπάνω, το pipeline εκτελείται αυτόματα από άκρη σε άκρη: καθαρίζει το CSV, παράγει το RDF, το φορτώνει στην GraphDB με reasoning και εκτελεί τα έξι SPARQL ερωτήματα τυπώνοντας τα αποτελέσματα.

10. Συμπεράσματα

Το υλοποιημένο τμήμα της εργασίας προσφέρει ένα πλήρες, αυτοματοποιημένο pipeline σε Java που καλύπτει όλα τα ζητούμενα στάδια: από την προεπεξεργασία των ακατέργαστων δεδομένων, στον μετασχηματισμό τους σε RDF μέσω RML κανόνων, στη φόρτωσή τους με reasoning στην GraphDB, και τελικά στην εκτέλεση μη τετριμμένων SPARQL ερωτημάτων.

Ο διαχωρισμός σε αυτόνομες κλάσεις καθιστά το pipeline ευανάγνωστο, ελέγξιμο και επεκτάσιμο. Δύο μετασχηματισμοί υλοποιήθηκαν προγραμματιστικά ως συμπλήρωμα των RML κανόνων - ο καθαρισμός του CSV και η μετατροπή της οντολογίας σε Turtle - διότι αφορούσαν διορθώσεις και μετατροπές μορφής που δεν εκφράζονταν αποδοτικά σε επίπεδο RML. Η ενεργοποίηση του

reasoning, σε συνδυασμό με τα κατάλληλα ερωτήματα, αναδεικνύει με σαφήνεια τα πλεονεκτήματα της αναπαράστασης των δεδομένων σε RDF και της χρήσης μιας domain οντολογίας.